

LẬP TRÌNH CHO TRÍ TUỆ NHÂN TẠO

Giảng viên: Nguyễn Ngọc Giang

Tóm tắt nội dung bài trước

- Python đòi hỏi lập trình viên triệt để xử lý các vấn đề phát sinh khi thực thi chương trình bằng cơ chế ngoại lệ
- Python cung cấp cú pháp **try-except-else-finally** để xử lý ngoại lệ
 - Khối **try** chứa đoạn mã có thể phát sinh lỗi
 - Khối **except** để xử lý ngoại lệ phát sinh từ khối **try**
 - Khối **else** thực thi trong trường hợp khối **try** không sinh lỗi
 - Khối **finally** luôn được thực thi trong mọi tình huống, sử dụng để thực thi những đoạn mã “dọn dẹp” các vấn đề còn tồn đọng
- Lập trình viên có thể tự sinh ngoại lệ bằng cách sử dụng lệnh **raise** và có thể tự tạo hệ thống ngoại lệ của riêng mình nếu cần

Nội dung hôm nay: Làm việc với tập tin

1. Xử lý tập tin của Python
2. Đóng/Mở tập tin
3. Đọc/Ghi dữ liệu của tập tin
4. Con trỏ tập tin
5. Làm việc với hệ thống thư mục
6. Ví dụ trên một vài loại dữ liệu
7. Bài tập

Phần 1

Xử lý tập tin của Python

Xử lý tập tin của Python

- Tập tin (file) và thư mục (folder) là thành phần cơ bản của hệ thống lưu trữ dữ liệu bền vững
 - Tuy có một vài hệ thống không sử dụng những khái niệm này
- Python cung cấp khả năng xử lý tập tin như là thành phần cơ bản của ngôn ngữ (một số ngôn ngữ lập trình khác xem xử lý tập tin là tính năng mở rộng)
- Python chia các tác vụ tập tin làm hai nhóm:
 1. **Tác vụ quản lý**: không ảnh hưởng đến nội dung tập tin (đổi tên, di chuyển, xóa, sao chép, phân quyền,...)
 2. **Tác vụ nội dung**: có tương tác với nội dung tập tin (đọc, ghi,...)
- Loại tác vụ quản lý, Python cung cấp nhiều hàm thuộc thư viện `os` (`import os`) để xử lý chỉ với 1-2 dòng lệnh

Xử lý tập tin của Python

- Loại tác vụ nội dung, Python thực hiện quy trình 3 bước **mở tập tin – xử lý – đóng tập tin** tương tự như các ngôn ngữ lập trình khác
- Python cũng chia tập tin làm hai loại:
 - **Tập tin văn bản**: chứa nội dung chủ yếu là text và các dấu trình bày (tab, xuống dòng, căn lề,...)
 - Python tự động xử lý việc lưu trữ dấu xuống dòng khác nhau giữa các hệ điều hành Linux/Unix và Windows
 - Python hỗ trợ việc tự động chuyển đổi mã hóa (encode) giữa các loại văn bản khác nhau
 - **Tập tin nhị phân**: Python xem như dãy các byte dữ liệu và thường thao tác theo các khối dữ liệu để tăng tốc độ xử lý

Phần 2

Đóng/Mở tập tin

Làm việc với nội dung tập tin

- Làm việc với nội dung tập tin trong python gồm 3 bước:
 1. **Mở tập tin**: Đây là bước yêu cầu hệ thống chuẩn bị các điều kiện cần thiết để đọc/ghi nội dung tập tin bao gồm định vị dữ liệu trên vùng lưu trữ và khởi tạo các vùng đệm
 2. **Làm việc với tập tin**: Bước chính của quá trình, trong bước này chương trình thực hiện các thao tác liên quan đến nội dung tập tin
 3. **Đóng tập tin**: Đảm bảo nội dung mới được cập nhật lên vùng lưu trữ và giải phóng các tài nguyên đã được cấp phát để làm việc với tập tin
- Các bước này đều có thể phát sinh ngoại lệ **IOError**
- Đây là 3 bước chung của mọi ngôn ngữ lập trình khi làm việc với tập tin, không phải của riêng Python

Mở file

- Thao tác mở file khá chậm, vì thực hiện những bước sau:
 - Kiểm tra người dùng có mở quá nhiều file không
 - Kiểm tra file có tồn tại trên hệ thống hay không
 - Kiểm tra chương trình có quyền truy cập nội dung hay không
 - Kiểm tra có thể thao tác file vào thời điểm hiện tại hay không
 - File có thể bị khóa bởi chương trình khác
 - File có thể chỉ đọc vì được ghi trên thiết bị cấm ghi
 - File có thể chỉ ghi vì nó là loại thiết bị cấm đọc
 - Định vị vùng dữ liệu file trên thiết bị lưu trữ
 - Chuẩn bị vùng đệm cho việc đọc/ghi dữ liệu
- Vì vậy chỉ mở file khi cần thiết
- Chọn cách mở file phù hợp với mục đích xử lý

Mở file: `f = open(filename, mode)`

Giá trị	Ý nghĩa
r	Mở tập tin văn bản chỉ để đọc
r+	Mở tập tin văn bản để đọc và ghi
rb	Mở tập tin nhị phân chỉ để đọc
rb+, r+b	Mở tập tin nhị phân để đọc và ghi
w	Mở tập tin văn bản để ghi, nếu tập tin không tồn tại thì sẽ tạo mới
w+	Mở tập tin văn bản để đọc và ghi, nếu tập tin không tồn tại thì sẽ tạo mới
wb	Mở tập tin nhị phân để ghi, nếu tập tin không tồn tại thì sẽ tạo mới
wb+, w+b	Mở tập tin nhị phân để đọc và ghi, nếu tập tin không tồn tại thì sẽ tạo mới
a	Mở tập tin văn bản để ghi tiếp vào cuối nếu tập tin đã tồn tại, nếu tập tin không tồn tại thì sẽ tạo mới

Mở file: `f = open(filename, mode)`

Giá trị	Ý nghĩa
a+	Mở tập tin văn bản để đọc và ghi tiếp vào cuối nếu tập tin đã tồn tại, nếu tập tin không tồn tại thì sẽ tạo mới
ab	Mở tập tin nhị phân để ghi tiếp vào cuối nếu tập tin đã tồn tại, nếu tập tin không tồn tại thì sẽ tạo mới
ab+, a+b	Mở tập tin nhị phân để đọc và ghi tiếp vào cuối nếu tập tin đã tồn tại, nếu tập tin không tồn tại thì sẽ tạo mới
x	Tạo tập tin văn bản mới để ghi, sinh lỗi nếu tập tin đã tồn tại
x+	Tạo tập tin văn bản mới để đọc và ghi, sinh lỗi nếu tập tin đã tồn tại
xb	Tạo tập tin nhị phân mới để ghi, sinh lỗi nếu tập tin đã tồn tại
xb+, x+b	Tạo tập tin nhị phân mới để đọc và ghi, sinh lỗi nếu tập tin đã tồn tại
b	Mở tập tin nhị phân để đọc
t	Mở tập tin văn bản để đọc (đây là giá trị mặc định của mode khi gọi hàm open)

Ví dụ về mở file

mở file mode 'r' hoặc 'rt' để đọc

```
f1 = open("test.txt")
```

mở file mode 'w' để ghi

```
f2 = open("test.txt", 'w')
```

mở file mode 'r+b' để đọc và ghi dạng nhị phân

```
f3 = open("img.bmp", 'r+b')
```

mở file văn bản để đọc

chỉ định rõ nội dung được mã hóa dạng utf-8

```
f4 = open("test.txt", mode = 'r', encoding = 'utf-8')
```

Đóng file: `f.close()`

- Thao tác đóng file rất quan trọng:
 - Đẩy mọi dữ liệu trên vùng đệm xuống thiết bị lưu trữ
 - Cập nhật thông tin trên hệ thống file (filesize, last update,...)
 - Giải phóng vùng dữ liệu dùng cho làm việc với file
- Vì vậy khi không sử dụng đến file nữa, nên đóng file ngay
- Quên đóng file có gây lỗi không? Không có lỗi, hệ thống tự đóng tất cả các file đang mở khi kết thúc chương trình
- Sử dụng phát biểu `with` giúp tự động đóng file:

```
with open("test.txt", encoding = 'utf-8') as
```

```
    f:    # thực hiện các thao tác với tệp
```

```
    ...
```

```
# biến f bị hủy, tệp tin được tự động đóng lại
```

Đóng file: `f.close()`

- Cách làm đúng nhất là đóng file trong khối `finally`:

```
try:
```

```
    # mở tập tin
```

```
    f = open("test.txt", encoding = 'utf-8')
```

```
    # thực hiện các thao tác với tệp
```

```
    ...
```

```
finally:
```

```
    # đóng tập tin
```

```
    f.close()
```

- Chú ý: ngay cả lệnh đóng file cũng có thể sinh ngoại lệ, trong trường hợp này ngoại lệ sẽ được trả về khối `try` bên ngoài

Phần 3

Đọc/Ghi dữ liệu của tập tin

Các hàm đọc file

- **read(N)**: đọc N byte tiếp theo
 - Nếu không viết N thì đọc đến cuối file
 - Nếu dữ liệu trong file không đủ N byte thì đọc đến cuối file
 - Nếu tập tin mở ở chế độ văn bản thì trả về **str**
 - Nếu tập tin mở ở chế độ nhị phân thì trả về dãy **byte**
- **readline(N)**: đọc một dòng từ file, tối đa là N byte, nếu không viết N thì trả về str là dữ liệu được đọc tới khi nào gặp kí tự hết dòng hoặc hết file
 - Dữ liệu trả về bao gồm cả kí tự xuống dòng \n, trừ tình huống đọc dòng cuối cùng của file

Các hàm đọc file

- `readlines(N)`: sử dụng `readline` đọc các dòng cho đến hết file và trả về một danh sách các string, nếu viết `N` thì sẽ xử lý tối đa là `N` byte
- Các lập trình viên Python có cách đọc mọi dòng của file đơn giản hơn rất nhiều:

```
for line in open("test.txt", encoding = 'utf-8'):  
    # thực hiện các thao tác với từng dòng  
    ...
```

- Như vậy file văn bản trong Python là một kiểu tuần tự
- Đoạn mã trên có thể ghép cùng với phát biểu `with`:

```
with open('workfile') as f:  
    for line in f:  
        print(line, end='')
```

Các hàm ghi dữ liệu ra file

- **write(data)**: ghi data ra file, trả về số byte ghi được
 - Phương thức làm việc với cả file văn bản và file nhị phân
 - Nếu file văn bản thì data phải là kiểu **str**
 - Nếu file nhị phân thì data phải là khối byte (kiểu **bytearray** hoặc kiểu **bytes**)
- **writelines(data)**: ghi toàn bộ nội dung data vào file theo từng dòng
 - Chỉ làm việc với kiểu file văn bản
 - Dữ liệu data phải là danh sách các str
 - Nếu cố dùng kiểu dữ liệu khác sẽ phát sinh lỗi **TypeError**

Phần 4

Con trở tập tin

Con trỏ tập tin

- Con trỏ tập tin là vị trí hiện thời sẽ đọc/ghi dữ liệu, tương tự như khi ta ghi dữ liệu lên màn hình
- Một tập tin chỉ có một con trỏ tập tin
- Khi mở tập tin ở chế độ “thêm cuối” (a – append), con trỏ tập tin tự động đặt ở cuối tập tin
- Các chế độ mở tập tin khác luôn đặt con trỏ tập tin ở đầu
- Python cung cấp một số lệnh cho phép lấy vị trí và di chuyển con trỏ tập tin
 - Chỉ nên sử dụng với tập tin nhị phân
 - Không phải loại tập tin nào cũng lấy được vị trí hoặc dịch chuyển được con trỏ

Phương thức làm việc với con trỏ tập tin

Phương thức	Chức năng
seek(offset, from)	Chuyển con trỏ tập tin đến vị trí mới offset, tham số thứ hai from quyết định cách tính vị trí: - SEEK_SET = 0: tính từ đầu tập tin (mặc định) - SEEK_CUR = 1: tính từ vị trí hiện thời - SEEK_END = 2: tính từ cuối tập tin
seekable()	Trả về True nếu tập tin là dạng truy cập ngẫu nhiên (dùng được phương thức seek ở trên)
tell()	Trả về vị trí con trỏ tập tin hiện tại (tính từ đầu tập tin)

Các phương thức khác của tập tin

Phương thức	Chức năng
fileno()	Trả về một số nguyên là mã định danh của tập tin
flush()	Đẩy dữ liệu khỏi vùng đệm, ghi xuống thiết bị lưu trữ
isatty()	Trả về True nếu tập tin được kết nối với một thiết bị đầu cuối
read(N)	Đọc N byte tiếp theo của tập tin
readable()	Trả về True nếu tập tin có thể đọc được
readline()	Đọc và trả về một dòng từ tập tin
readlines()	Đọc và trả về một danh sách các dòng từ tập tin
truncate(size)	Cắt tập tin hiện tại lấy đúng thành size byte đầu tiên
writable()	Trả về True nếu tập tin có thể ghi được.
write(S)	Ghi nội dung biến S xuống tập tin
writelines(lines)	Ghi nội dung biến lines xuống tập tin, biến này là một danh sách các chuỗi, các chuỗi sẽ được ghi liên tiếp xuống tập tin

Phần 5

Làm việc với hệ thống thư mục

Làm việc với hệ thống thư mục

- Các phương thức quản lý tập tin và thư mục đều thuộc thư viện `os` (`import os`)
- Các phương thức loại này đều có thể sinh lỗi `OSError`
- Để tương thích giữa các hệ điều hành, Python xem các dấu gạch chéo đều là dấu phân tách đường dẫn
 - Nghĩa là `"c:\\test"`, `r'c:\test'` và `"c:/test"` được coi là như nhau
- Lấy thư mục làm việc hiện tại: `os.getcwd()`
 - Trường hợp lấy str dạng mã ASCII thì dùng `os.getcwdb()`
- Thay đổi thư mục làm việc: `os.chdir(path)` với `path` là đường dẫn đến thư mục mới
- Tạo thư mục mới: `os.mkdir(path)`

Làm việc với hệ thống thư mục

- Lấy danh sách các thư mục và tập tin nằm trong thư mục path: `os.listdir(path)`
 - Nếu bỏ tham số `path` thì lấy danh sách từ thư mục hiện tại
- Đổi tên thư mục hoặc tập tin: `os.rename(old, new)`
 - Phương thức làm việc với cả tập tin và thư mục
- Xóa bỏ tập tin: `os.remove(filename)`
- Xóa bỏ thư mục: `os.rmdir(path)`
 - Phương thức chỉ xóa được thư mục trống
 - Trường hợp muốn xóa thư mục bất kể nó trống hay không, có thể sử dụng phương thức `shutil.rmtree(path)` (cần `import` thư viện `shutil`)

Phân giải đường dẫn theo thư mục làm việc

Chú ý: Python phân giải đường dẫn theo thư mục hiện tại

Vì vậy cần rất cẩn thận khi viết đường dẫn trong các lệnh

```
import os
```

tạo thư mục abc trong thư mục hiện tại

```
os.mkdir('abc')
```

tạo thư mục xyz trong thư mục abc thuộc thư mục hiện tại

```
os.mkdir('abc/xyz')
```

tạo thư mục abc trong thư mục temp thuộc ổ đĩa C

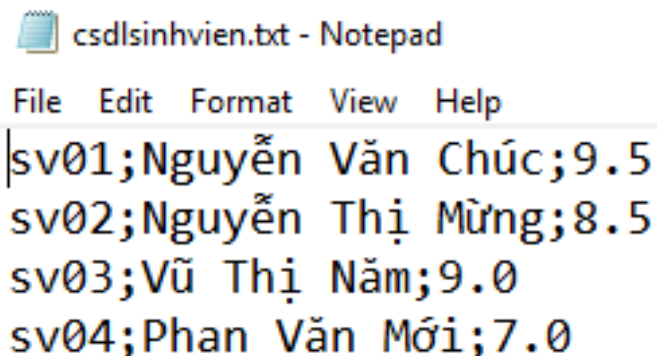
```
os.mkdir('C:/temp/abc')
```

Phần 6

Ví dụ

Ví dụ file text

```
1 def luuFile():
2     file=open('csdlsinhvien.txt', 'w', encoding='utf-8')
3     file.writelines("sv01;Nguyễn Văn Chúc;9.5\n")
4     file.writelines("sv02;Nguyễn Thị Mừng;8.5\n")
5     file.writelines("sv03;Vũ Thị Năm;9.0\n")
6     file.writelines("sv04;Phan Văn Mới;7.0")
7     file.close()
8     luuFile()
```



csdlsinhvien.txt - Notepad

File Edit Format View Help

sv01;Nguyễn Văn Chúc;9.5
sv02;Nguyễn Thị Mừng;8.5
sv03;Vũ Thị Năm;9.0
sv04;Phan Văn Mới;7.0

Ví dụ file text

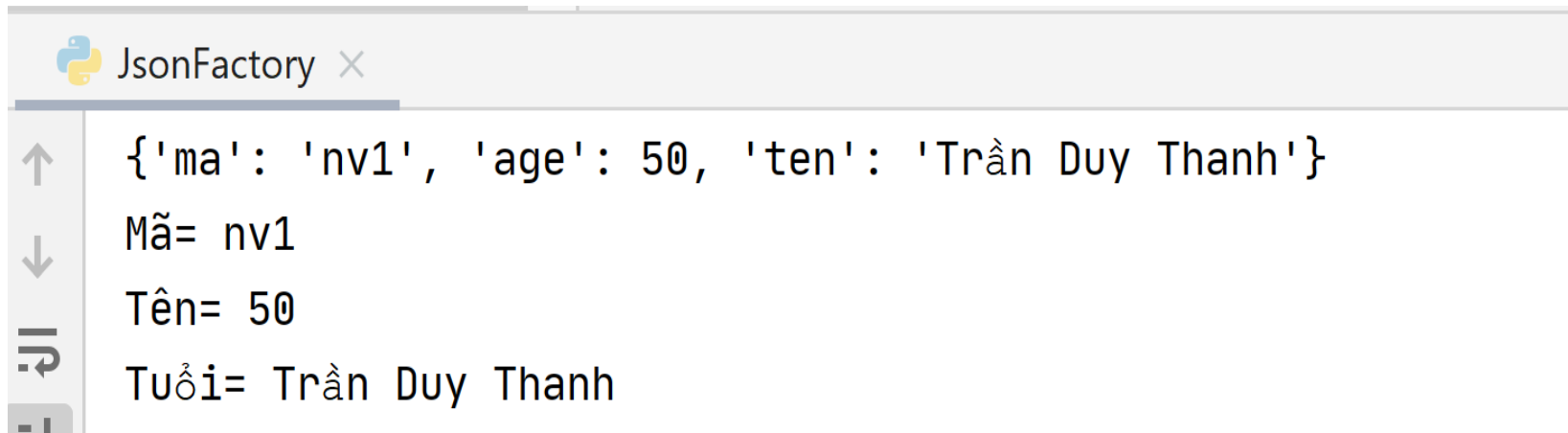
Cách đọc tập tin

```
def docFile():  
    file = open('csdlsinhvien.txt', 'r', encoding='utf-8')  
    for line in file:  
        print(line.strip())  
    file.close()  
docFile()
```

Ví dụ file JSON

```
import json
jsonString = '{ "ma":"nv1", "age":50, "ten":"Trần Duy Thanh"}'
dataObject=json.loads(jsonString)
print(dataObject)
print("Mã=",dataObject["ma"])
print("Tên=",dataObject["age"])
print("Tuổi=",dataObject["ten"])
```

Chuyển đổi String
Json qua Python
Object



```
JsonFactory ×
↑ {'ma': 'nv1', 'age': 50, 'ten': 'Trần Duy Thanh'}
↓ Mã= nv1
Tên= 50
↻ Tuổi= Trần Duy Thanh
```

Ví dụ file JSON

```
import json
```

```
pythonObject = {  
    "ten": "Trần Duy Thanh",  
    "tuoi": 50,  
    "ma": "nv1"  
}
```

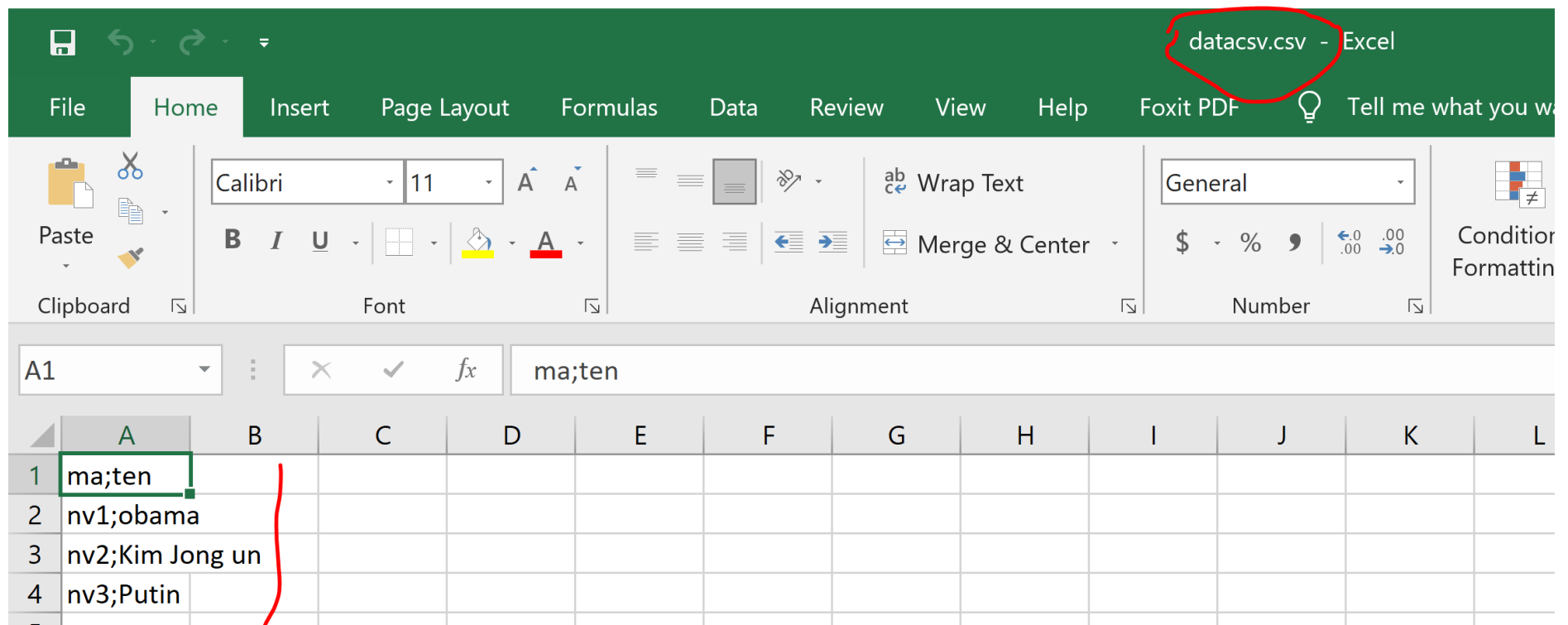
```
jsonString = json.dumps(pythonObject)
```

```
print(jsonString)
```

Chuyển đổi
Python Object
qua Json String

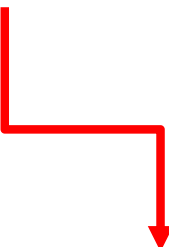
Ví dụ file CSV

File CSV với mỗi cột ngăn cách bởi dấu ;



Ví dụ file CSV

```
import csv
with open('datacsv.csv', newline='') as f:
    reader = csv.reader(f, delimiter=';', quoting=csv.QUOTE_NONE)
    for row in reader:
        print(row[0], "\t", row[1])
```

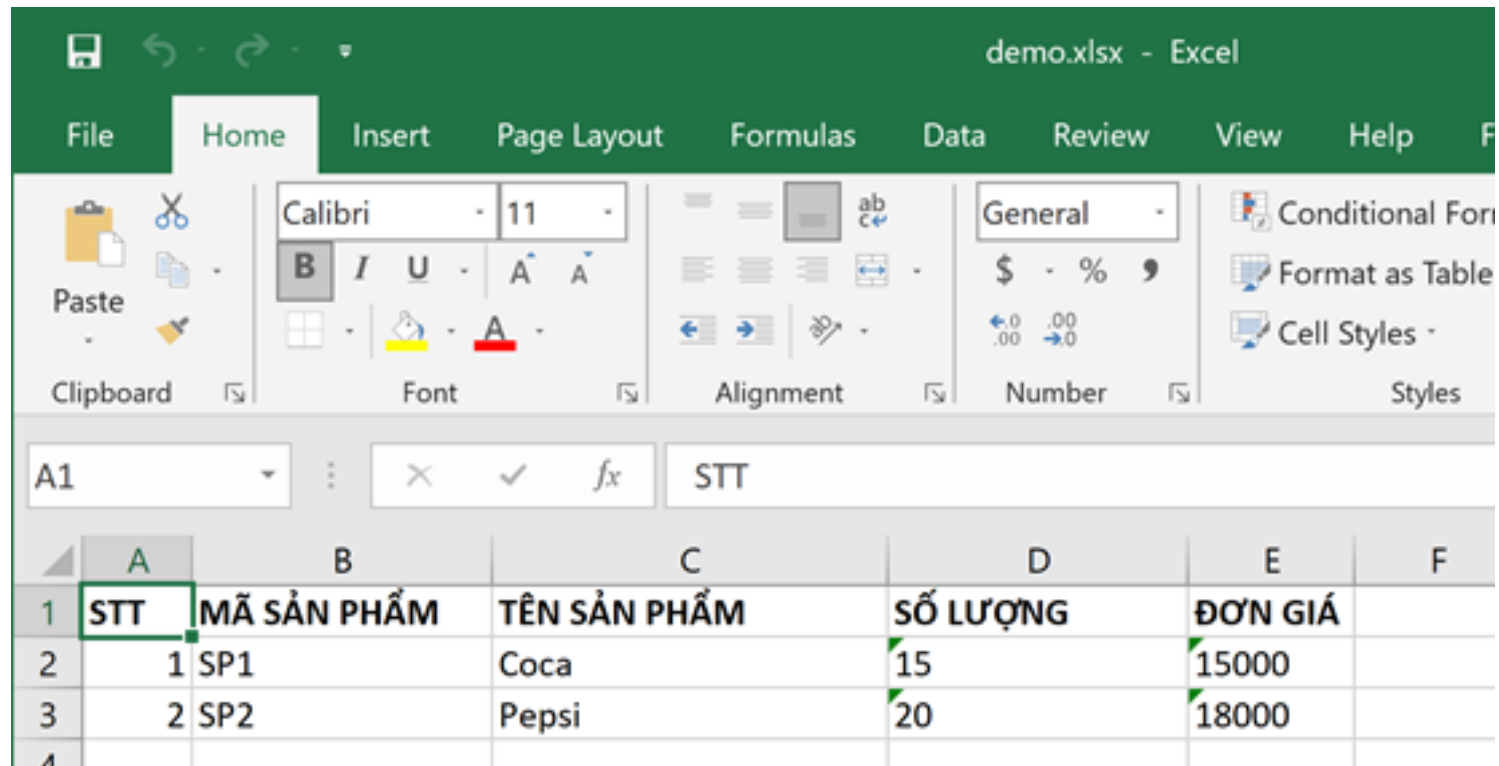


The screenshot shows a Python IDE window titled "CSVFactory". The command line at the top indicates the script is running from "D:/Python/FileFactory/CSVFactory.py". The console output shows the execution of the script, displaying the contents of the CSV file in a tab-separated format:

ma	ten
nv1	obama
nv2	Kim Jong un
nv3	Putin

Ví dụ file Excel

Dùng Python để tạo ra file Excel có định dạng dưới đây



The screenshot shows the Microsoft Excel interface with the 'demo.xlsx' file open. The 'Home' tab is selected, showing various formatting options like font, alignment, and number. The active cell is A1, which contains the text 'STT'. Below the ribbon, a table is displayed with the following data:

	A	B	C	D	E	F
1	STT	MÃ SẢN PHẨM	TÊN SẢN PHẨM	SỐ LƯỢNG	ĐƠN GIÁ	
2	1	SP1	Coca	15	15000	
3	2	SP2	Pepsi	20	18000	
4						

Ví dụ file Excel

- Cài đặt thư viện: `xlsxwriter`
- Link: <https://xlsxwriter.readthedocs.io/>

```
import xlsxwriter
```

Ví dụ file Excel

```
import xlswriter
```

```
# Tạo một file excel cùng 1 sheet
```

```
workbook = xlswriter.Workbook('demo.xlsx')  
worksheet = workbook.add_worksheet()
```

```
# thiết lập các cột cho file
```

```
worksheet.set_column('A:A', 5)  
worksheet.set_column('B:B', 15)  
worksheet.set_column('C:C', 20)  
worksheet.set_column('D:D', 15)  
worksheet.set_column('E:E', 15)
```

```
# định dạng tiêu đề cột in đậm
```

```
bold = workbook.add_format({'bold': True})
```

```
# thêm dòng tiêu đề và định dạng in đậm
```

```
worksheet.write('A1', 'STT', bold)  
worksheet.write('B1', 'MÃ SẢN PHẨM', bold)  
worksheet.write('C1', 'TÊN SẢN PHẨM', bold)  
worksheet.write('D1', 'SỐ LƯỢNG', bold)  
worksheet.write('E1', 'ĐƠN GIÁ', bold)
```

```
#thêm một dòng dữ liệu
```

```
worksheet.write('A2', 1)  
worksheet.write('B2', 'SP1')  
worksheet.write('C2', 'Coca')  
worksheet.write('D2', '15')  
worksheet.write('E2', '15000')
```

```
#thêm một dòng dữ liệu
```

```
worksheet.write('A3', 2)  
worksheet.write('B3', 'SP2')  
worksheet.write('C3', 'Pepsi')  
worksheet.write('D3', '20')  
worksheet.write('E3', '18000')
```

```
#Chèn Logo vào
```

```
worksheet.insert_image('B5', 'logo_UEL.png')
```

```
workbook.close()
```

Chạy ct và vào thư mục chứa ct xem file Excel sẽ có kết quả như mong muốn

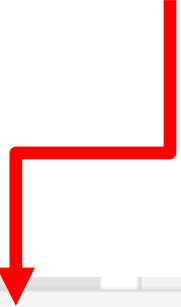
Ví dụ file Excel

- Để đọc file excel, có thể dùng: Openpyxl
- Link: <https://openpyxl.readthedocs.io/en/stable/>

```
import openpyxl
```

Ví dụ file Excel

```
from openpyxl import load_workbook
wb = load_workbook('demo.xlsx')
print(wb.sheetnames)
ws = wb[wb.sheetnames[0]]
for row in ws.values:
    for value in row:
        print(value, "\t", end="")
    print("")
```



Run: ExcelOpenpyxl x

C:\Python39\python.exe D:/Python/FileFactory/ExcelOpenpyxl.py

['Sheet1']

STT	MÃ SẢN PHẨM	TÊN SẢN PHẨM	SỐ LƯỢNG	ĐƠN GIÁ
1	SP1	Coca	15	15000
2	SP2	Pepsi	20	18000

Phần 7

Bài tập

Bài tập

1. Đọc 1 file và in ra màn hình 5 dòng đầu tiên, nếu file không đủ 5 dòng thì in toàn bộ nội dung file.
2. Đọc 1 file và in ra màn hình 5 dòng cuối cùng, nếu file không đủ 5 dòng thì in toàn bộ nội dung file.
3. Đọc 1 file, tìm và in ra nội dung của dòng dài nhất trong file đó
4. Đọc 1 file, tìm và in ra từ dài nhất trong file
5. Đọc 1 file, thống kê và in ra tất cả các chữ cái có trong file và số lần xuất hiện của các chữ đó
6. Đọc 1 file, thống kê và tần xuất xuất hiện của tất cả các từ trong file, in theo thứ tự giảm dần của số lần xuất hiện

Bài tập

7. Đọc tập tin văn bản abc.txt sau đó chuyển nội dung đọc được sang chữ in hoa và ghi vào tập tin xyz.txt
8. Đọc tập tin văn bản abc.txt, lọc lấy tất cả các số trong dữ liệu đọc được và ghi chúng lần lượt vào tập tin number.txt, mỗi số ghi trên một dòng
9. Nhập tên của hai tập tin văn bản, đọc và so sánh xem nội dung của hai tập tin có giống nhau hay không
10. Lấy danh sách các tập tin trong thư mục hiện tại, tiến hành so sánh nội dung của các tập tin trong thư mục, và chỉ ra các tập tin có nội dung giống nhau. Chương trình in ra các nhóm tập tin có cùng nội dung, đánh số các nhóm tăng dần bắt đầu từ 1

Bài tập

11. Ảnh jpg là một tập tin nhị phân có đặc điểm là 4 byte dữ liệu từ vị trí thứ 6 đến thứ 9 tạo thành chữ JFIF (vị trí bắt đầu tính từ số 0). Hãy nhập tên tập tin, kiểm tra xem đó có thể là tập tin ảnh jpg không?
12. Ảnh bitmap (BMP) là một tập tin nhị phân:
- 2 byte đầu tạo nên chữ BM
 - 4 byte từ vị trí 18-21 tạo nên số nguyên 4 byte là số dòng (chiều cao) của ảnh
 - 4 byte từ vị trí 22-25 tạo nên số nguyên 4 byte là số cột (chiều ngang) của ảnh
- Nhập tên tập tin, cho biết đó có phải là ảnh bitmap không, nếu đúng thì in ra cỡ của ảnh (dòng x cột)

Bài tập

13: Viết chương trình cho phép người dùng nhập vào thông tin của sinh viên như name, age, gender (giới tính) để xây dựng CSDL. Lưu thông tin vào file .txt hoặc .json hoặc .csv. Sau đó đọc file đã lưu và in ra thông tin của danh sách sinh viên đã nhập

14: Viết chương trình cho phép người dùng nhập vào thông tin của hóa đơn bán hàng bao gồm: product_code (mã sản phẩm), product_name (tên sản phẩm), quantity (số lượng), unit_price (đơn giá) để xây dựng CSDL. Lưu thông tin vào file .csv. Sau đó đọc file đã lưu và in ra danh sách hóa đơn (Trong CSDL bên trên cần có thêm thông tin như STT (số thứ tự), total_price (tổng giá tiền của đơn hàng)).